HTTP credentials

uv supports credentials over HTTP when querying package registries.

Authentication can come from the following sources, in order of precedence:

- The URL, e.g., `https://<user>:<password>@<hostname>/...`
- A netrc configuration file
- The uv credentials store
- A keyring provider (off by default)

Authentication may be used for hosts specified in the following contexts:

- `[index]`
- `index-url`
- `extra-index-url`
- `find-links`
- `package @ https://...`

netrc files

`.netrc` files are a long-standing plain text format for storing credentials on a system.

Reading credentials from `.netrc` files is always enabled. The target file path will be loaded from the `NETRC` environment variable if defined, falling back to `~/.netrc` if not.

The uv credentials store

uv can read and write credentials from a store using the `uv auth` commands.

Credentials are stored in a plaintext file in uv's state directory, e.g., `~/.local/share/uv/credentials/credentials.toml` on Unix. This file is currently not intended to be edited manually.

!!! note

```
A secure, system native storage mechanism is in [preview](../preview.md) — it is
still
experimental and being actively developed. In the future, this will become the
default storage
mechanism.

When enabled, uv will use the secret storage mechanism native to your operating
system. On
macOS, it uses the Keychain Services. On Windows, it uses the Windows Credential
Manager. On
Linux, it uses the DBus-based Secret Service API.

Currently, uv only searches the native store for credentials it has added to the
secret store —
it will not retrieve credentials persisted by other applications.

Set `UV_PREVIEW_FEATURES=native-auth` to use this storage mechanism.
```

Keyring providers

A keyring provider is a concept from `pip` allowing retrieval of credentials from an interface matching the popular keyring Python package.

The "subprocess" keyring provider invokes the `keyring` command to fetch credentials. uv does not support additional keyring provider types at this time.

Set `--keyring-provider subprocess`, `UV_KEYRING_PROVIDER=subprocess`, or `tool.uv.keyring-provider = "subprocess"` to use the provider.

Persistence of credentials

If authentication is found for a single index URL or net location (scheme, host, and port), it will be cached for the duration of the command and used for other queries to that index or net location. Authentication is not cached across invocations of uv.

When using `uv add`, uv *will not* persist index credentials to the `pyproject.toml` or `uv.lock`. These files are often included in source control and distributions, so it is generally unsafe to include credentials in them. However, uv *will* persist credentials for direct URLs, i.e., `package @ https://username:password:example.com/foo.whl`, as there is not currently a way to otherwise provide those credentials.

If credentials were attached to an index URL during `uv add`, uv may fail to fetch dependencies from indexes which require authentication on subsequent operations. See the index authentication documentation for details on persistent authentication for indexes.

Learn more

See the index authentication documentation for details on authenticating index URLs.

See the `pip` compatibility guide for details on differences from `pip`.